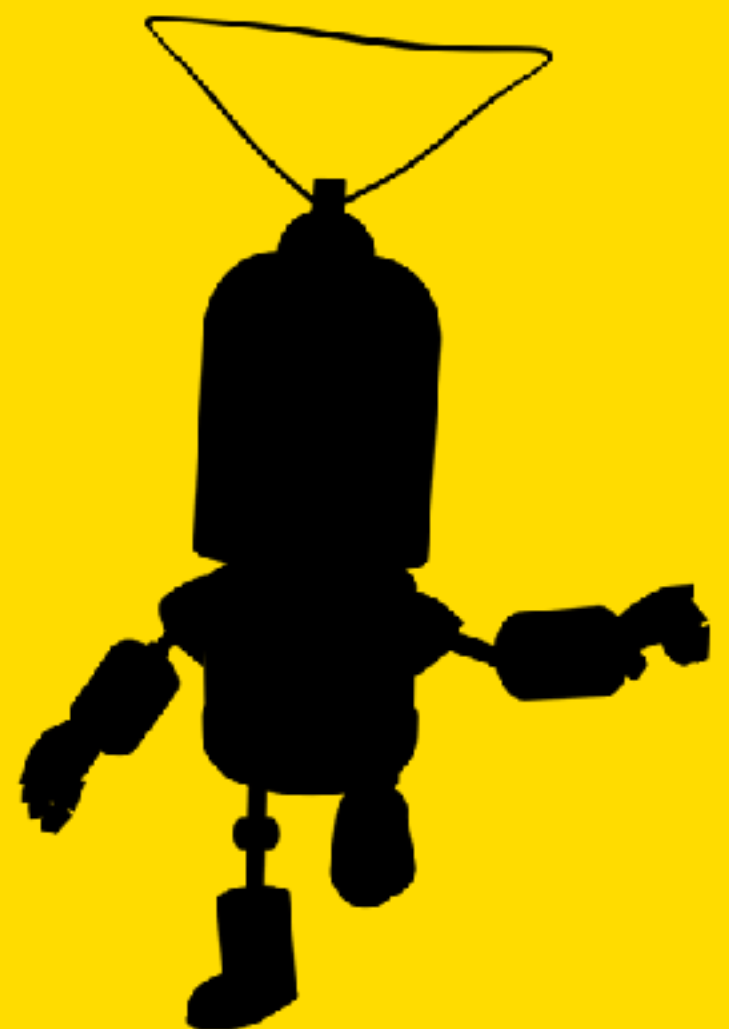


Reasoning with uncertainty

COMP9414 Artificial Intelligence

Oliver Obst



Overview

1	5/6	Introduction. Agents & search		
2	12/6	Planning		release assignment #1
3	19/6	Knowledge representation		
4	26/6	Neural networks		
5	3/7	Reinforcement learning		deadline assignment #1 (6 Jul)
6	10/7			release assignment #2
→ 7	17/7	Reasoning with uncertainty		
8	24/7	Computer vision		
9	31/7	Foundation models		deadline assignment #2 (31 Jul)
10	7/8	Human-aligned intelligent robotics		

Reasoning with uncertainty

Slides material and acknowledgement:

- CS50AI Week 2 uncertainty material for probability vocabulary, conditional probability, Bayes' rule, random variables, and Bayes nets.
- Berkeley CS188 Bayes net notes for representation, factorisation, independence intuitions, inference, decision networks, and value of information.
- UW CSE571 robotics material for state estimation, noisy sensors, noisy actions, and Bayes-filter belief updates.
- scikit-learn probability calibration documentation for reliability diagrams and calibrated classifier probabilities.
- The ai9414 uncertainty demo for the office localisation belief-state explorer.

After this lecture

you should be able to

- explain why uncertainty is unavoidable in AI systems;
- compute a simple Bayesian update using Bayes' rule;
- distinguish prior, likelihood, evidence, and posterior;
- describe a Bayesian network as a compact representation of a joint distribution;
- explain conditional independence at an intuitive level;
- interpret a belief distribution over possible robot states;
- explain how expected utility links uncertain belief to action;
- explain why a neural network confidence score is not automatically a reliable probability;
- describe calibration and abstention as uncertainty-related ideas in modern AI.

What do we add today?

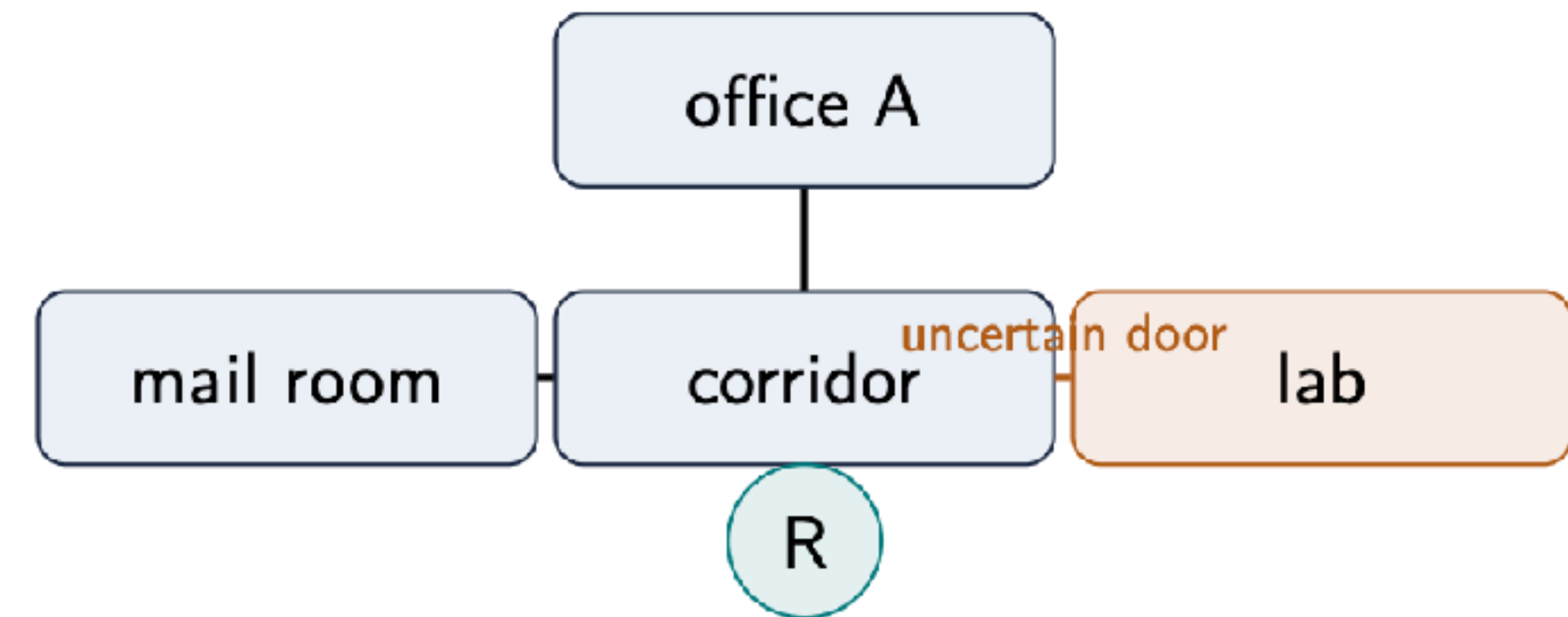
Earlier simplifying assumption	Most real systems
The agent knows the current state	The agent has noisy observations
Actions have known effects	Actions may fail or have variable outcomes
Facts are true or false	Evidence supports beliefs to different degrees
A model outputs an answer	The answer may be wrong, uncertain, or poorly calibrated

This week makes an hidden assumption explicit: intelligent agents rarely know the true state of the world.

Robot in a building example

A robot is asked to deliver an item to a lab. It must reason about:

- whether a door is open;
- whether a room is occupied;
- whether an object detector is correct;
- whether its localisation estimate is reliable;
- whether it is safe to continue.



What should the robot believe, and what should it do?

Four questions

1. Belief

What should I believe given uncertain evidence?

2. Structure

How can I represent dependencies between uncertain variables?

3. Decision

What should I do when my belief is uncertain?

4. Confidence

When a learned model says 0.9, should I trust that as 90%?

Noise or ignorance?

Two sources of uncertainty:

Aleatoric uncertainty

Noise or randomness in the world.

Examples: noisy sensors, wheel slip, blurred images.

Epistemic uncertainty

Limited knowledge, data, or model coverage.

Example: an object type the robot has never seen.

More data can reduce epistemic uncertainty. Better sensing can reduce some aleatoric uncertainty, but not remove all of it.

A probability is a belief over alternatives

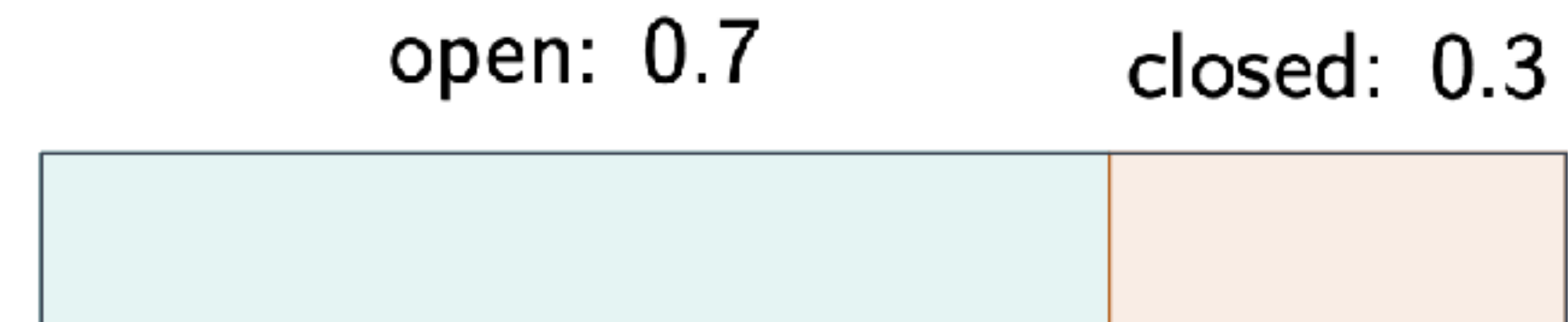
The robot might assign:

$$P(\text{DoorOpen}) = 0.7$$

$$P(\text{DoorClosed}) = 0.3$$

This does not mean the door is partly open.

It means the robot does not know the true state.



A belief distribution assigns probability mass across mutually exclusive possibilities.

Random variables

A **random variable** is a named uncertain quantity with possible values.

Binary variables

DoorOpen $\in \{\text{true}, \text{false}\}$

PersonPresent $\in \{\text{true}, \text{false}\}$

DetectorSaysPerson $\in \{\text{true}, \text{false}\}$

BatteryLow $\in \{\text{true}, \text{false}\}$

Multi-valued variables

Location $\in \{\text{mail}, \text{office A}, \text{corridor}, \text{office B}, \text{lab}\}$

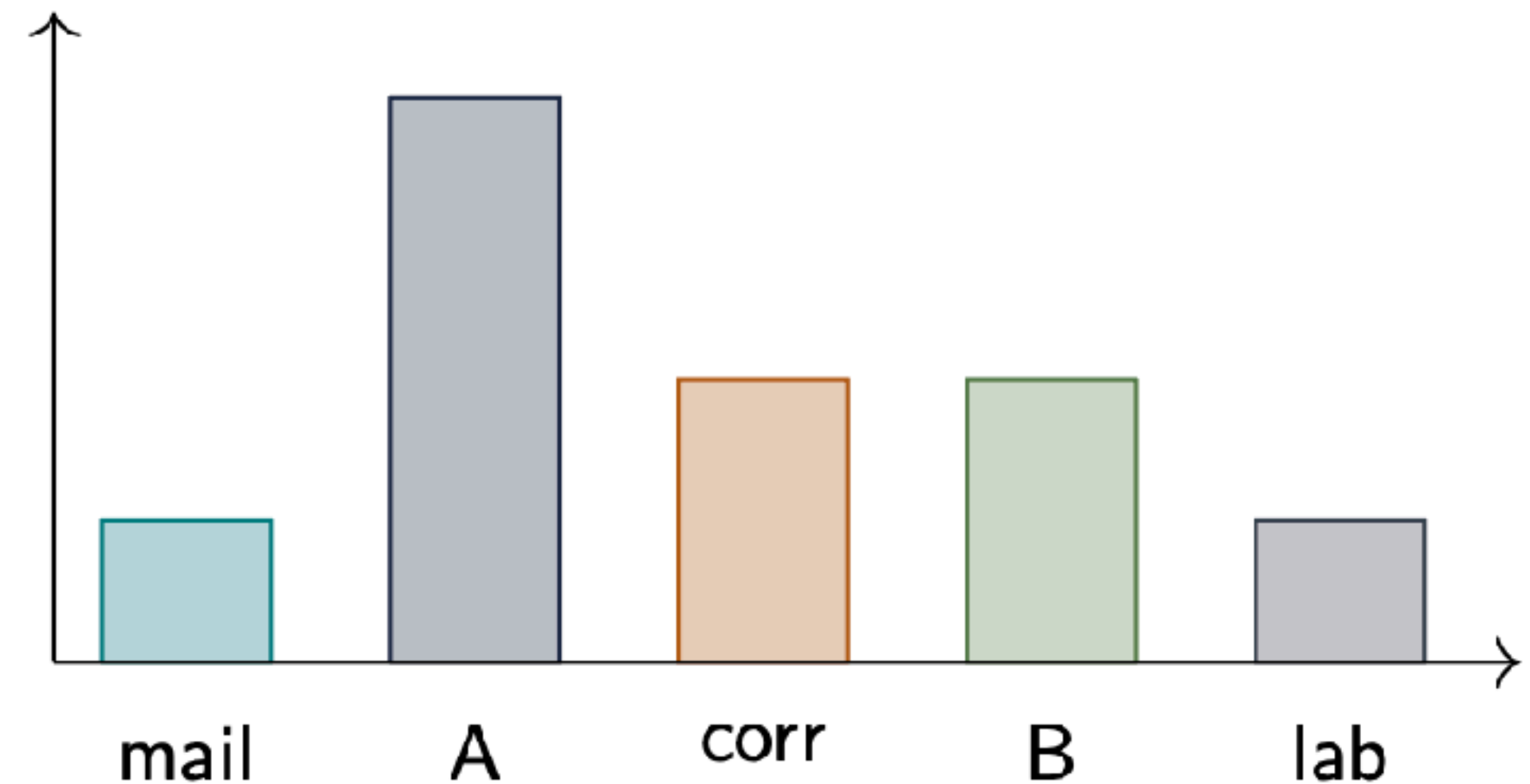
ObjectClass $\in \{\text{cup}, \text{ornament}, \text{unknown}\}$

Distributions must sum to one

A distribution gives one probability for each possible value.

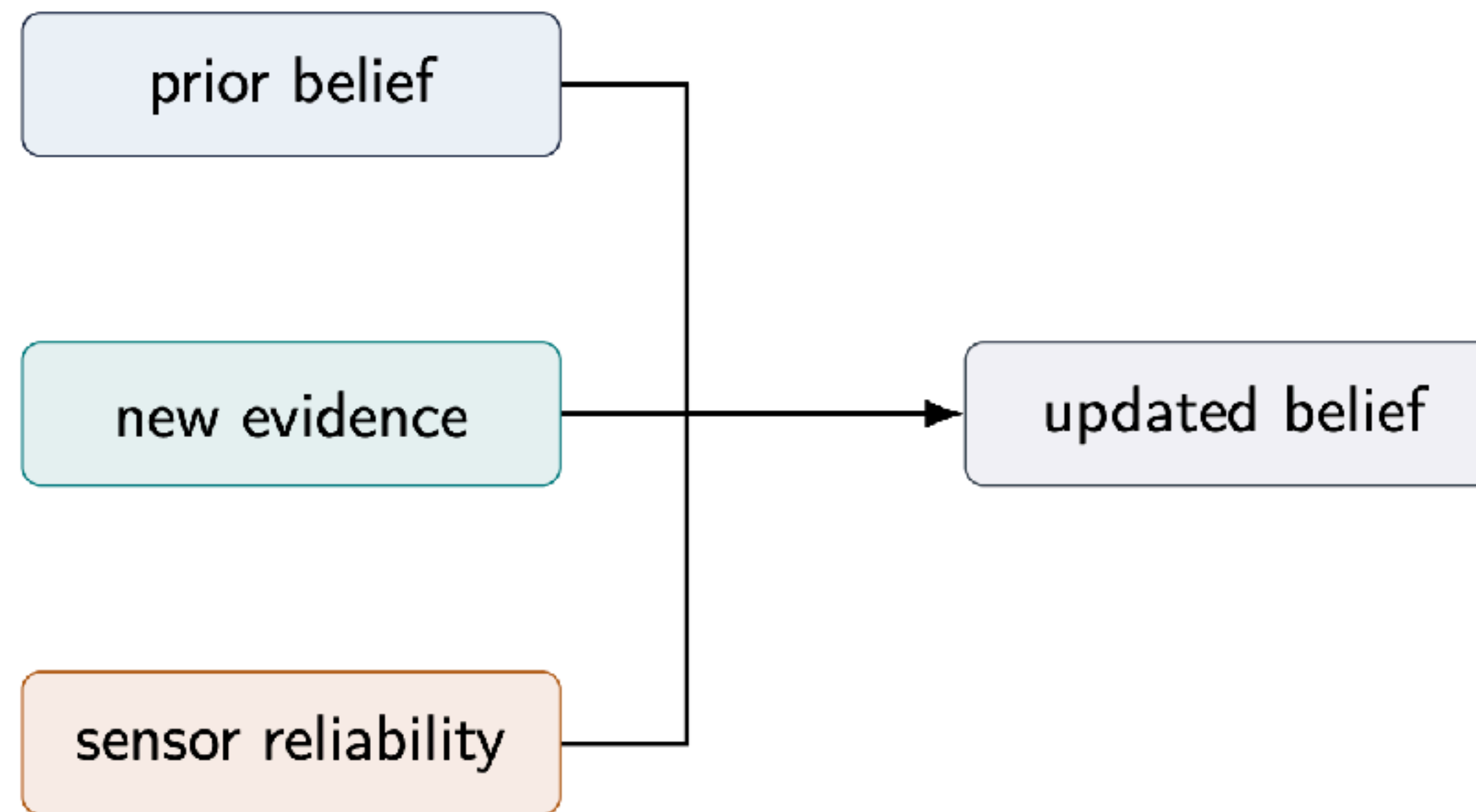
$$\sum_x P(\text{location} = x) = 1$$

x	mail	A	corridor	B	lab
P(x)	0.1	0.4	0.2	0.2	0.1



The state itself is hidden; the distribution is what the agent carries.

Prior, evidence, likelihood, posterior



Prior: belief before the new observation.

Evidence: the observation that has arrived.

Likelihood: how probable that observation is under each possible state.

Posterior: belief after updating with the evidence.

Conditional probability

The bar means “given”:

$$P(\text{DetectorSaysOccupied} \mid \text{RoomOccupied})$$

This is not the same as:

$$P(\text{RoomOccupied} \mid \text{DetectorSaysOccupied})$$

Sensors are often specified in the first direction, but agents often need the second direction.

Independence and conditional independence

Independence

Knowing A tells me nothing about B.

$$P(A, B) = P(A)P(B)$$

Conditional independence

Once I know C, knowing A tells me nothing extra about B.

$$P(A, B | C) = P(A | C) P(B | C)$$

Conditional independence is often the modelling assumption that makes probabilistic AI practical.

Are these sensors independent?

A camera and a lidar both report an obstacle in front of the robot.
Which answer is best?

- A. They must be independent because they are different sensors.
- B. They must be dependent because they saw the same object.
- C. It depends on the modelling assumptions and failure modes.
- D. Independence is only relevant for logic, not probability.

Are these sensors independent?

Correct answer: **C**.

The two readings might be treated as independent if their errors are unrelated.

They may be dependent if:

- they are responding to the same underlying object;
- both fail under glass, rain, darkness, glare, or reflective surfaces;
- one sensor reading is processed using the other sensor's output.

Independence is not a property you get for free. It is a claim about the model.

The problem Bayes' rule solves

The robot wants:

$$P(\text{DoorOpen} \mid \text{SensorSaysOpen})$$

But what it usually knows is:

$$P(\text{SensorSaysOpen} \mid \text{DoorOpen})$$

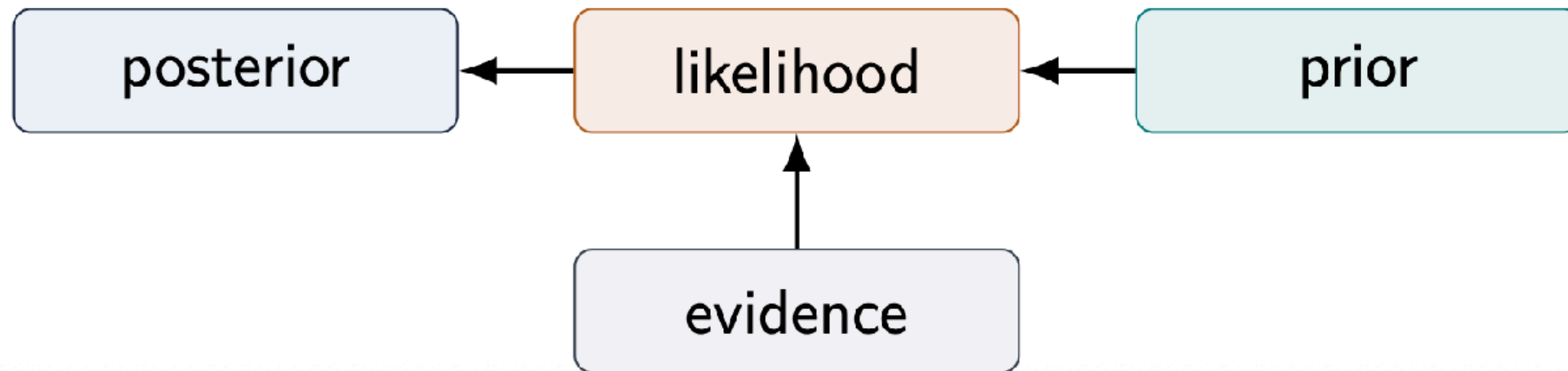
$$P(\text{SensorSaysOpen} \mid \text{DoorClosed})$$

$$P(\text{DoorOpen}), P(\text{DoorClosed})$$

Bayes' rule lets the agent reverse the direction of the conditional probability.

Bayes' rule

$$P(H|E) = \frac{P(E|H) P(H)}{P(E)}$$



$$\text{posterior} = \frac{\text{likelihood} \cdot \text{prior}}{\text{evidence}}$$

Binary normalisation form

For a binary hypothesis H , this version is often easiest by hand:

$$P(H|E) = \frac{P(E|H)P(H)}{P(E|H)P(H) + P(E|\neg H)P(\neg H)}$$

- The numerator is the unnormalised support for H .
- The denominator adds the support for all possible cases.
- Normalisation makes the posterior probabilities sum to one.

Example: Door sensor

Prior

$$P(\text{DoorOpen}) = 0.3$$

$$P(\text{DoorClosed}) = 0.7$$

Question

$$P(D | S) = ?$$

Sensor model Let $D = \text{DoorOpen}$
and $S = \text{SensorOpen}$.

$$P(S | D) = 0.9$$

$$P(S | \neg D) = 0.2$$

The sensor is quite reliable, and the prior says the door is usually closed.

Door sensor calculation

$$\begin{aligned} P(\text{DoorOpen} \mid \text{SensorOpen}) &= \frac{0.9 \cdot 0.3}{0.9 \cdot 0.3 + 0.2 \cdot 0.7} \\ &= \frac{0.27}{0.27 + 0.14} \\ &= \frac{0.27}{0.41} \\ &\approx 0.66 \end{aligned}$$

A reliable sensor alone does not automatically imply near certainty.
The prior is also a part of this.

Base-rate trap

Keep the same sensor and change only the prior.

Case A

$$P(D) = 0.3$$

$$P(D | S) \approx 0.66$$

Case B

$$P(D) = 0.8$$

$$\frac{0.9 \cdot 0.8}{0.9 \cdot 0.8 + 0.2 \cdot 0.2} = \frac{0.72}{0.76} \approx 0.95$$

Same sensor, but different context: different posterior.

Bayes' rule is not magic

Bayes' rule is a coherent update rule, not a guarantee that the model is correct.

It is only as good as:

- the prior;
- the likelihoods;
- the modelling assumptions;
- the evidence actually observed;
- the decision process that uses the posterior.

“Bad probabilities in” still give “bad probabilities out”.

Where do the numbers come from?

The probabilities in a model can come from:

- **Domain knowledge:** a domain expert estimated sensor reliability;
- **Data:** counts how often a detector is correct on labelled examples;
- **Calibration experiments:** compare confidence with empirical accuracy;
- **Assumptions:** treat two sensors as conditionally independent.

The model is useful only if these numbers are meaningful for the setting where it is used.

The world has many uncertain variables

Robot example variables:

- DoorOpen
- PersonPresent
- LightsOn
- MotionDetected
- CameraSeesPerson
- EntrySafe

If all six are binary, a full joint distribution has:

$$2^6 = 64$$

entries.

For 20 binary variables:

$$2^{20} = 1,048,576$$

entries.

Joint distributions are complete

A full joint distribution can answer any probability query about its variables.

BatteryLow	MotorFault	MovingSlowly	Probability
F	F	F	0.684
F	F	T	0.036
F	T	F	0.054
F	T	T	0.126
T	F	F	0.048
T	F	T	0.032
T	T	F	0.001
T	T	T	0.019

The problem is scale: the table grows exponentially with the number of variables.

The problem is modelling, not just arithmetic

Full joint approach

- List every combination of variables
- Assign one probability to each complete world
- Make sure whole table sums to one.
- Query by summing rows and normalising.

Why this breaks

- Too many entries
- Hard to estimate from data
- Hard to explain (to humans)
- Ignores useful structure in the domain.

We need structure

Many variables only depend directly on a few others.

Plausible dependency

MovingSlowly depends on
BatteryLow and MotorFault.

No direct dependency

CameraBlurred may not
depend directly on BatteryLow.

Bayesian networks use a graph to encode this direct-dependency structure.

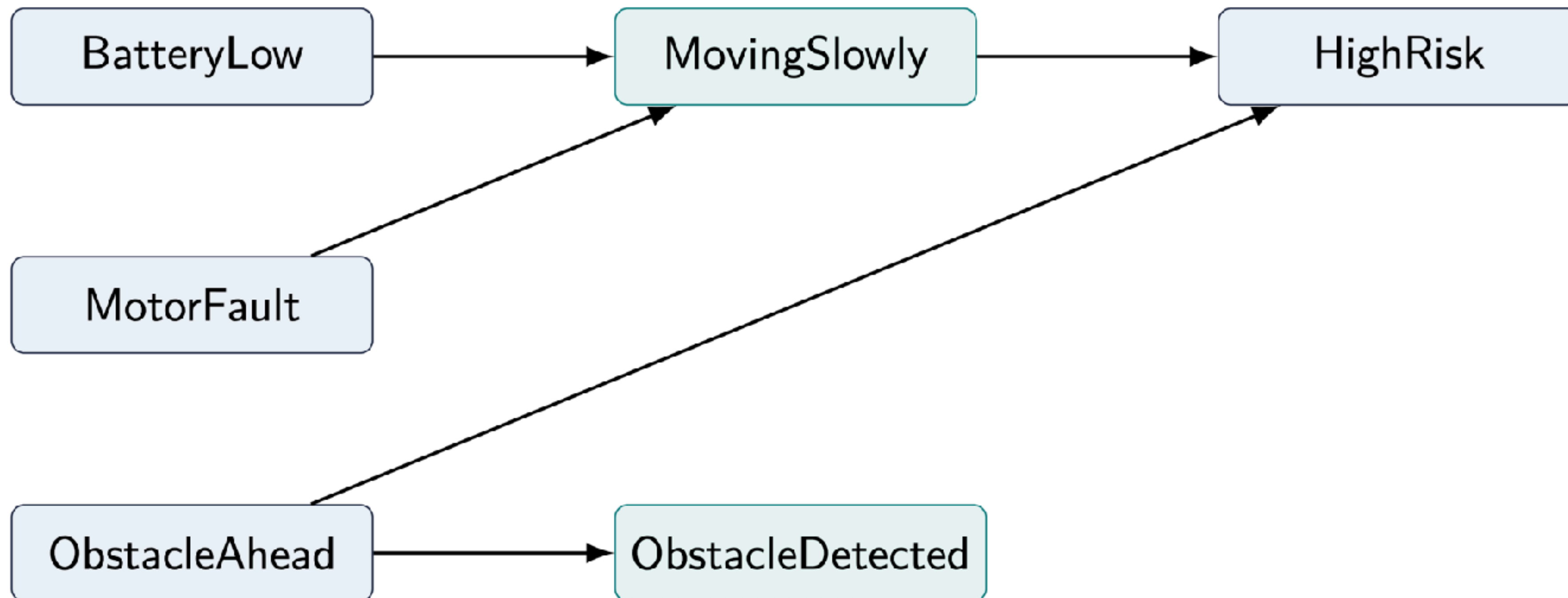
Bayesian Network: Graph plus local tables

A Bayesian Network consists of:

- A directed acyclic graph
- One random variable per node
- A conditional probability table for each node
- An assumption that each node depends directly only on its parents

The graph and local probability tables together define a compact joint distribution.

Robot diagnostic Bayes Net



The arrows say what variables are modelled as direct inputs to local probability tables.

Arrows may mean causes

A Bayes-net arrow means direct probabilistic dependence in the model.

It may be causal

Rain $\rightarrow$ CameraWet

But not always

Disease $\rightarrow$ Symptom

or

Symptom $\rightarrow$ Disease

can both appear in different predictive models.

For decision-making and intervention, causal meaning matters. For today, read the graph first as a compact probability model.

Local probability tables

Root node

$$P(\text{BatteryLow} = \text{true}) = 0.1$$

One parent node

O	$P(D = \text{true} \mid O)$
false	0.20
true	0.90

O = ObstacleAhead, D = ObstacleDetected.

Two parent node

B	M	$P(S=T \mid B, M)$
F	F	0.05
T	F	0.40
F	T	0.70
T	T	0.95

B = BatteryLow,
 M = MotorFault,
 S = MovingSlowly.

The full joint distribution factorises

General form

$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Pa}(X_i))$$

$\text{Pa}(X_i)$ means the parents of X_i in the graph.

For the robot diagnostic network:

$$P(B, M, S, O, D, H) = P(B) P(M) P(S \mid B, M) P(O) P(D \mid O) P(H \mid S, O)$$

B = BatteryLow, M = MotorFault, S = MovingSlowly, O = ObstacleAhead,
 D = ObstacleDetected, H = HighRisk.

Why factorisation helps

Naive joint table

6 binary variables $\Rightarrow 2^6 = 64$ rows

Every row needs a probability, and the rows must sum to one.

Bayes-net tables

- three root probabilities;
- one one-parent table;
- two two-parent tables.

The representation grows with parent sets, not with all variables at once.

What queries can we ask?

Let B = BatteryLow, M = MotorFault, S = MovingSlowly, O = ObstacleAhead, D = ObstacleDetected, and H = HighRisk.

Diagnostic

$$P(M | S)$$

Predictive

$$P(H | O)$$

Evidence combining

$$P(H | S, D)$$

Explaining

$$P(M | S, B)$$

Inference means computing probabilities of unknown variables given observed evidence.

Inference by summing cases

Suppose B = BatteryLow, M = MotorFault, S = MovingSlowly:

$$P(S = \text{true}) = \sum_b \sum_m P(S = \text{true} \mid b, m)P(b)P(m)$$

If $P(B) = 0.1$, $P(M) = 0.2$, and the S table is:

B	M	$P(S = \text{true} \mid B, M)$
F	F	0.05
T	F	0.40
F	T	0.70
T	T	0.95

Inference: calculation

$$\begin{aligned} P(S = \text{true}) &= 0.05 \cdot 0.9 \cdot 0.8 + 0.4 \cdot 0.1 \cdot 0.8 \\ &\quad + 0.7 \cdot 0.9 \cdot 0.2 + 0.95 \cdot 0.1 \cdot 0.2 \\ &= 0.036 + 0.032 + 0.126 + 0.019 \\ &= 0.213 \end{aligned}$$

We are summing over the hidden cases for the causes.

Exact Bayes-net inference algorithms organise this kind of summing and multiplying more efficiently than building the whole joint table.

Conditional independence in a chain

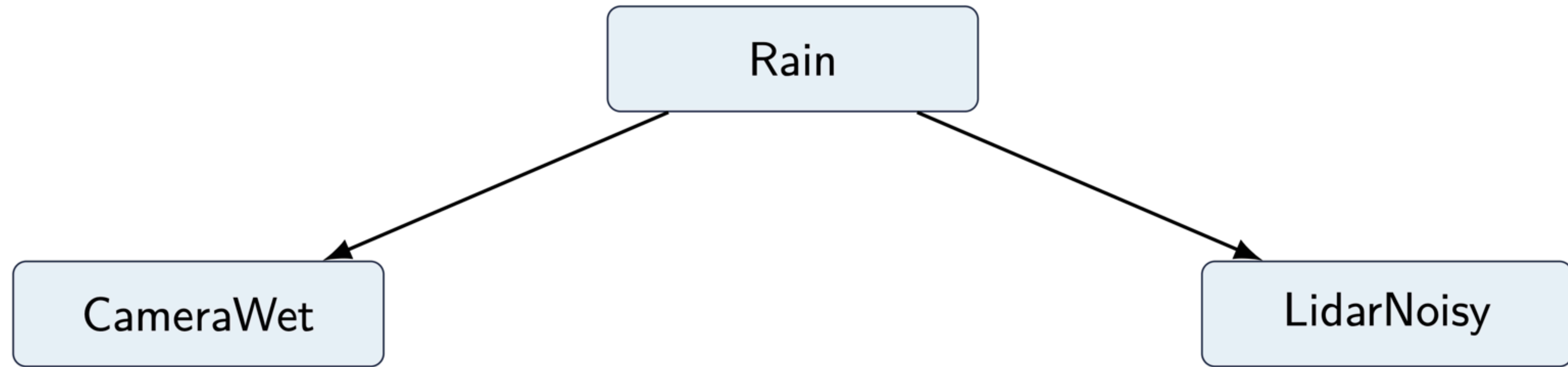


In this model, once we already know MovingSlowly, BatteryLow provides no extra direct information about HighRisk.

$$\text{HighRisk} \perp \text{BatteryLow} \mid \text{MovingSlowly}$$

This is a modelling assumption encoded by the graph. It may or may not be true in the real world.

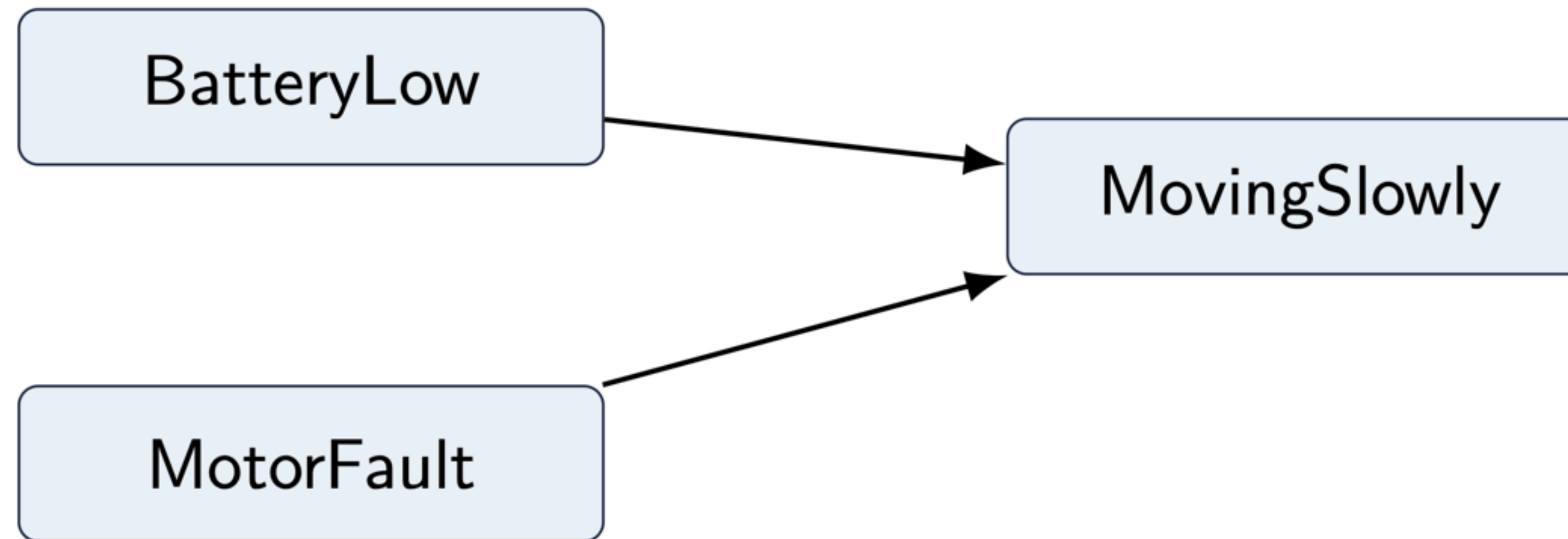
Common cause pattern



The observations may look related because they share a cause.

If we know whether it is raining, the remaining relation between the two sensor errors may be much weaker.

Explaining away



If the robot is moving slowly, both BatteryLow and MotorFault become more plausible.

If we then learn that the battery is definitely low, a motor fault becomes less necessary as an explanation.

Light D-separation intuition

D-separation is a graphical test for when a Bayes net implies conditional independence. (“d” – dependence)

For this lecture, we work with the intuition:

- A chain $A \rightarrow B \rightarrow C$ is blocked if B is known.
- A chain $A \leftarrow B \rightarrow C$ is blocked if B is known.
- A common effect $A \rightarrow B \leftarrow C$ behaves differently: observing B can make the causes dependent.

The important skill today is reading small graphs as modelling claims.

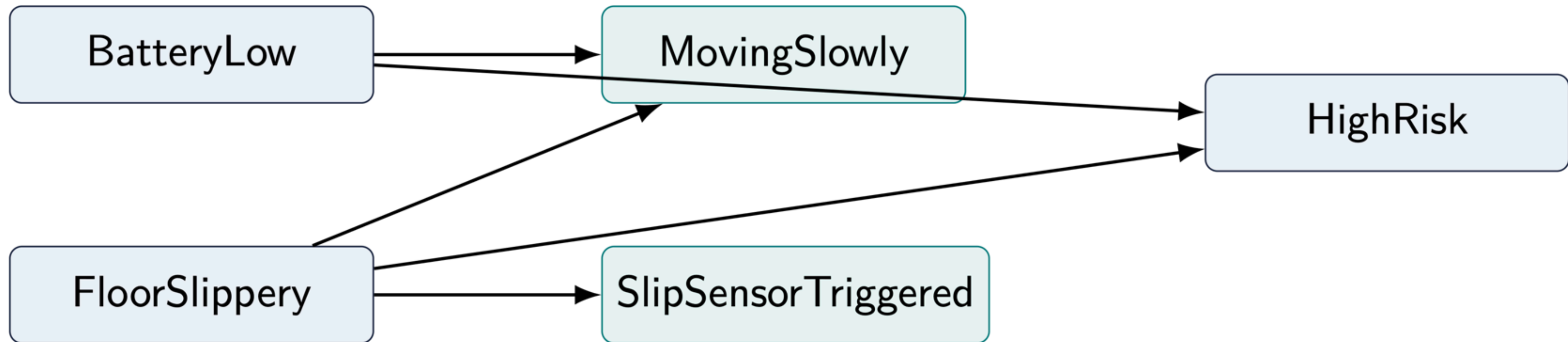
Creating a small Bayes net

Story:

A cleaning robot sometimes slows down. It may slow down because the battery is low or because the floor is slippery. A slip sensor is more likely to trigger when the floor is slippery, but sometimes triggers falsely. The corridor may be high risk if the floor is slippery or the battery is low.

- identify the random variables;
- draw a plausible Bayes net;
- mark observed and hidden variables;
- write one CPT row;
- explain what changes if the slip sensor triggers.

One plausible graph



If the only evidence were that the robot is moving slowly, a low battery would make the slippery-floor explanation less necessary. The slip sensor still gives direct evidence about the floor.

So far, belief has been mostly static.

Next part: belief over time:

Robots act, move, sense, and update repeatedly.

A robot that doesn't know where it is

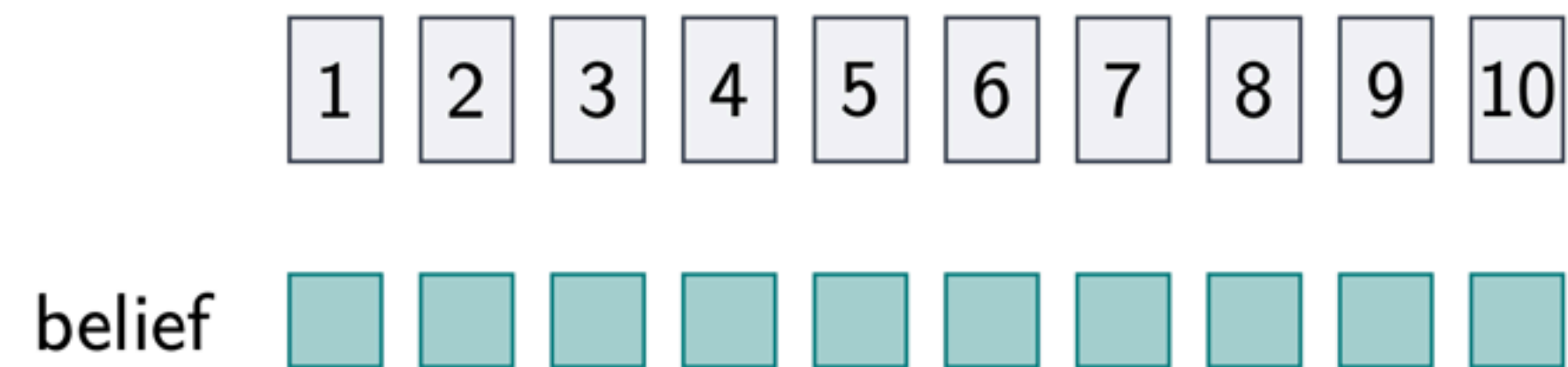
A mobile robot's exact location is unknown.

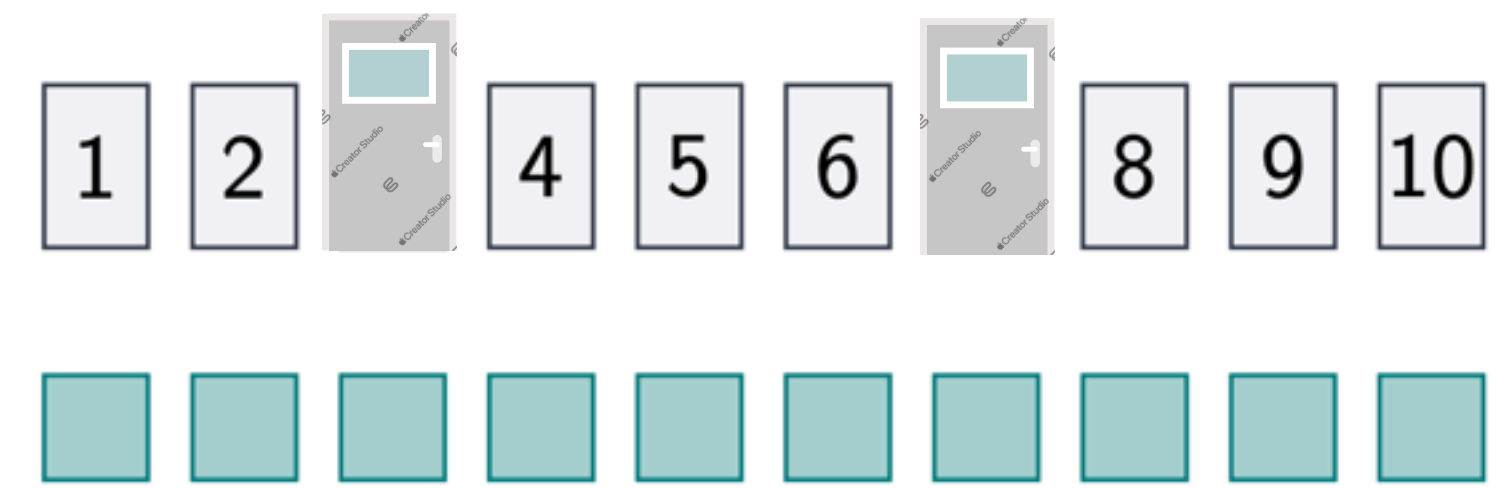
The robot maintains a belief distribution:

$P(\text{location} = 1), \dots, P(\text{location} = 10)$

$[0.1, 0.1, \dots, 0.1]$

Initially, it may not know much.





Sensor update

Suppose cells 3 and 7 have doors, and the robot sees a door.

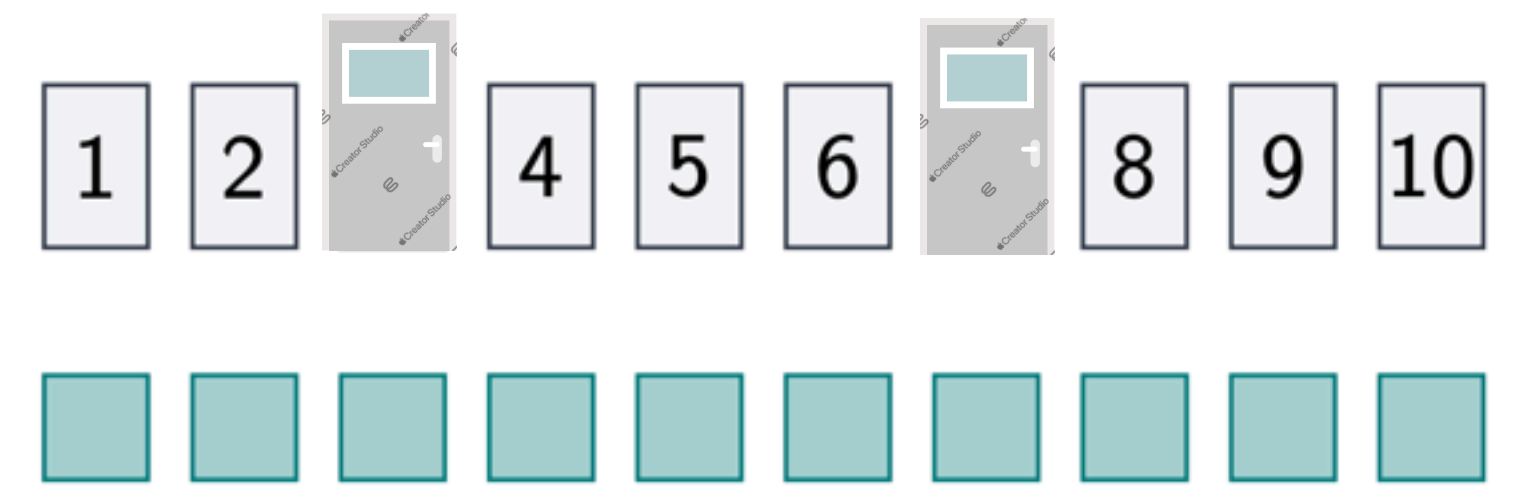
$$P(\text{SeesDoor} \mid \text{AtDoorCell}) = 0.8$$

$$P(\text{SeesDoor} \mid \text{NotAtDoorCell}) = 0.1$$

Update rule:

$$\text{new belief}(x) \propto P(\text{observation} \mid x) \cdot \text{old belief}(x)$$

Observations usually concentrate belief on states that make the observation likely.



Sensor update: numbers

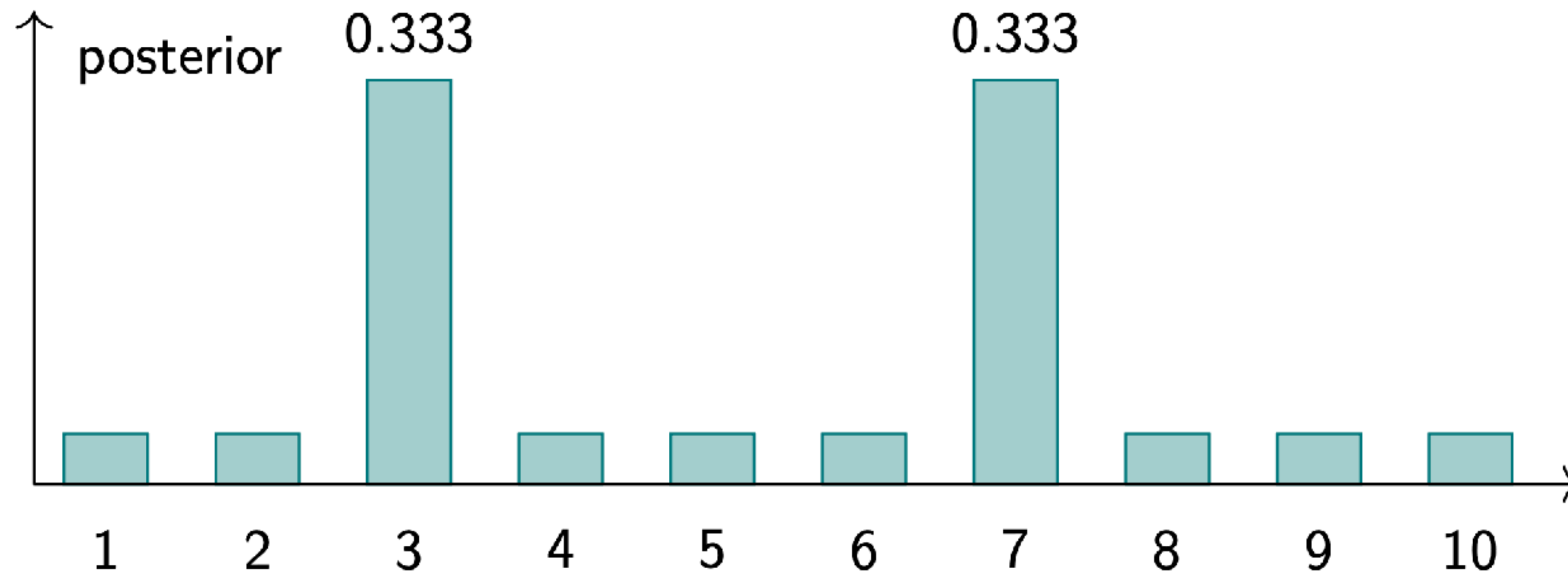
Door cells are 3 and 7. The prior is uniform, the robot sees a door.

x	1	2	3	4	5	6	7	8	9	10
old belief	.10	.10	.10	.10	.10	.10	.10	.10	.10	.10
sensor prob.	.10	.10	.80	.10	.10	.10	.80	.10	.10	.10
product	.01	.01	.08	.01	.01	.01	.08	.01	.01	.01
new belief	.042	.042	.333	.042	.042	.042	.333	.042	.042	.042

$$Z = \sum_x \text{product}(x) = 0.24, \quad \text{new belief}(x) = \frac{\text{product}(x)}{Z}$$

The sensor-probability row is $P(\text{SeesDoor} \mid X = x)$, not a belief over cells, so it does not need to sum to one. Dividing by Z turns the products into the new belief probabilities.

Sensor update: bar chart



Seeing a door makes cells 3 and 7 much more plausible, but it does not decide between them.

Motion update

Suppose the robot tries to move to the right.

$$P(\text{move right succeeds}) = 0.8$$

$$P(\text{stay in place}) = 0.1$$

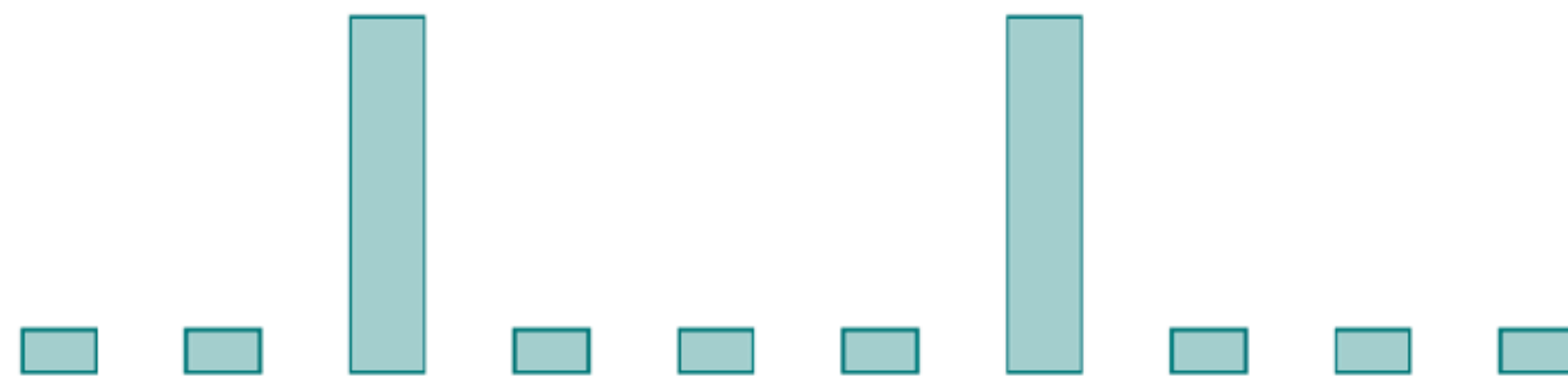
$$P(\text{move two cells}) = 0.1$$

A motion update predicts where the robot might be after an action.

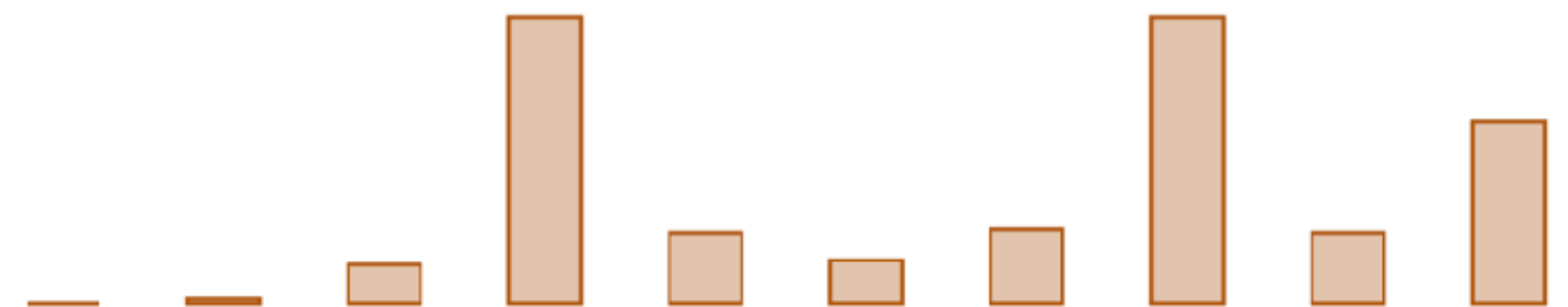
Uncertain actions usually shift and spread belief.

Motion update: shift and spread

Before action

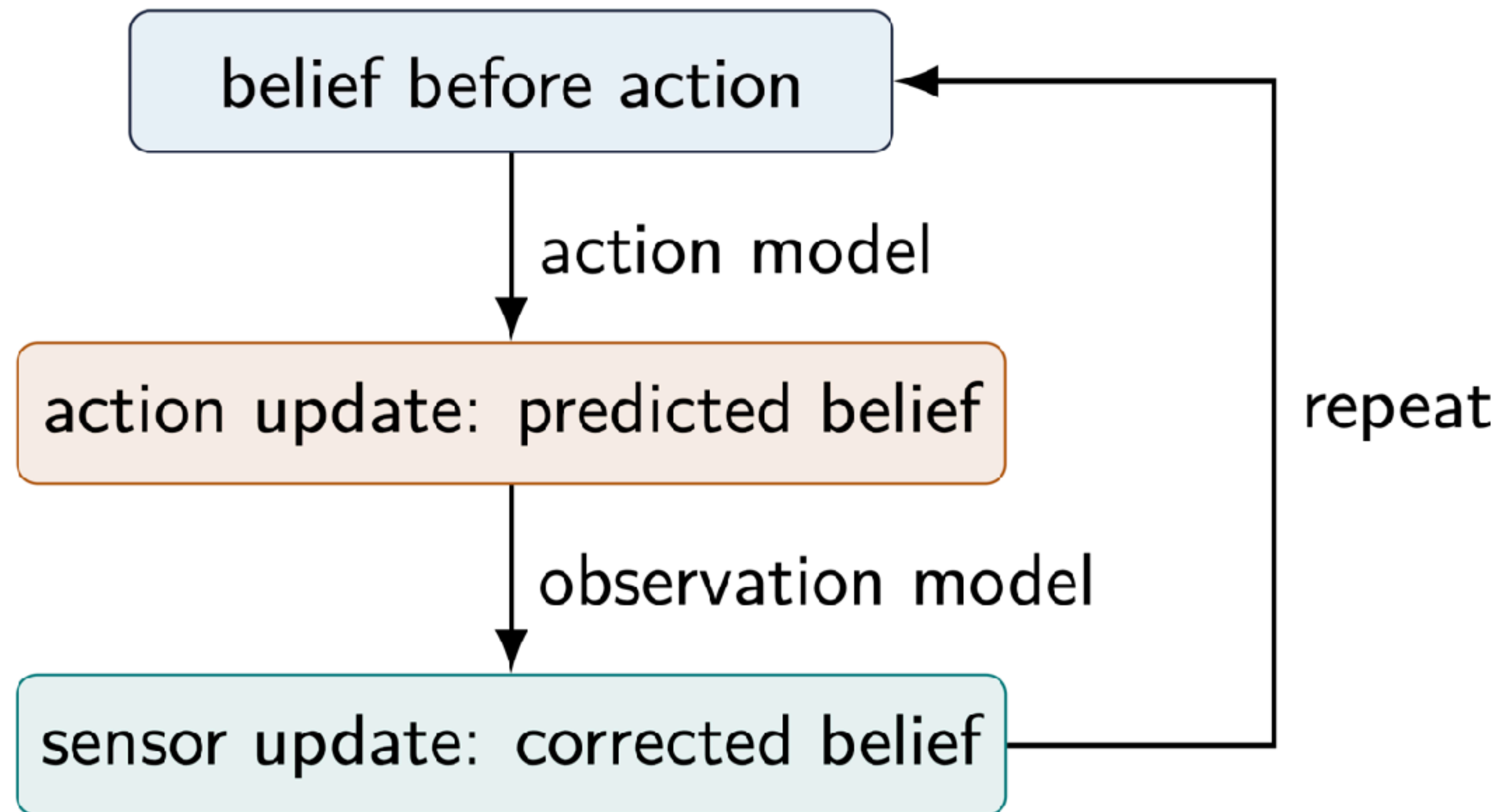


After noisy move right



The belief has moved, and uncertainty about action outcome has spread it across several cells.

Recursive belief update



Bayes filter apply this circle repeatedly.

Bayes filter summary formula

$$\underbrace{\text{bel}_t(x_t)}_{\text{posterior now}} = \eta \underbrace{P(z_t | x_t)}_{\text{sensor model}} \sum_{x_{t-1}} \underbrace{P(x_t | x_{t-1}, a_t)}_{\text{motion model}} \underbrace{\text{bel}_{t-1}(x_{t-1})}_{\text{previous belief}}$$

- Predict with the action model
- Correct with the observation model
- Normalise with η .

You do not need the formula to remember the core idea:
update belief, not just one guessed state.

Five cells Bayes filter

Initial belief

[0.2, 0.2, 0.2, 0.2, 0.2]

Door cells:

2 and 4

The robot sees a door:

$$P(\text{SeesDoor} \mid \text{DoorCell}) = 0.8$$

$$P(\text{SeesDoor} \mid \text{NonDoorCell}) = 0.2$$

Multiply by likelihoods, find the normalising constant, then normalise.

Five cells answer

x	1	2	3	4	5
prior	.2	.2	.2	.2	.2
likelihood	.2	.8	.2	.8	.2
unnormalised	.04	.16	.04	.16	.04

$$Z = 0.44$$

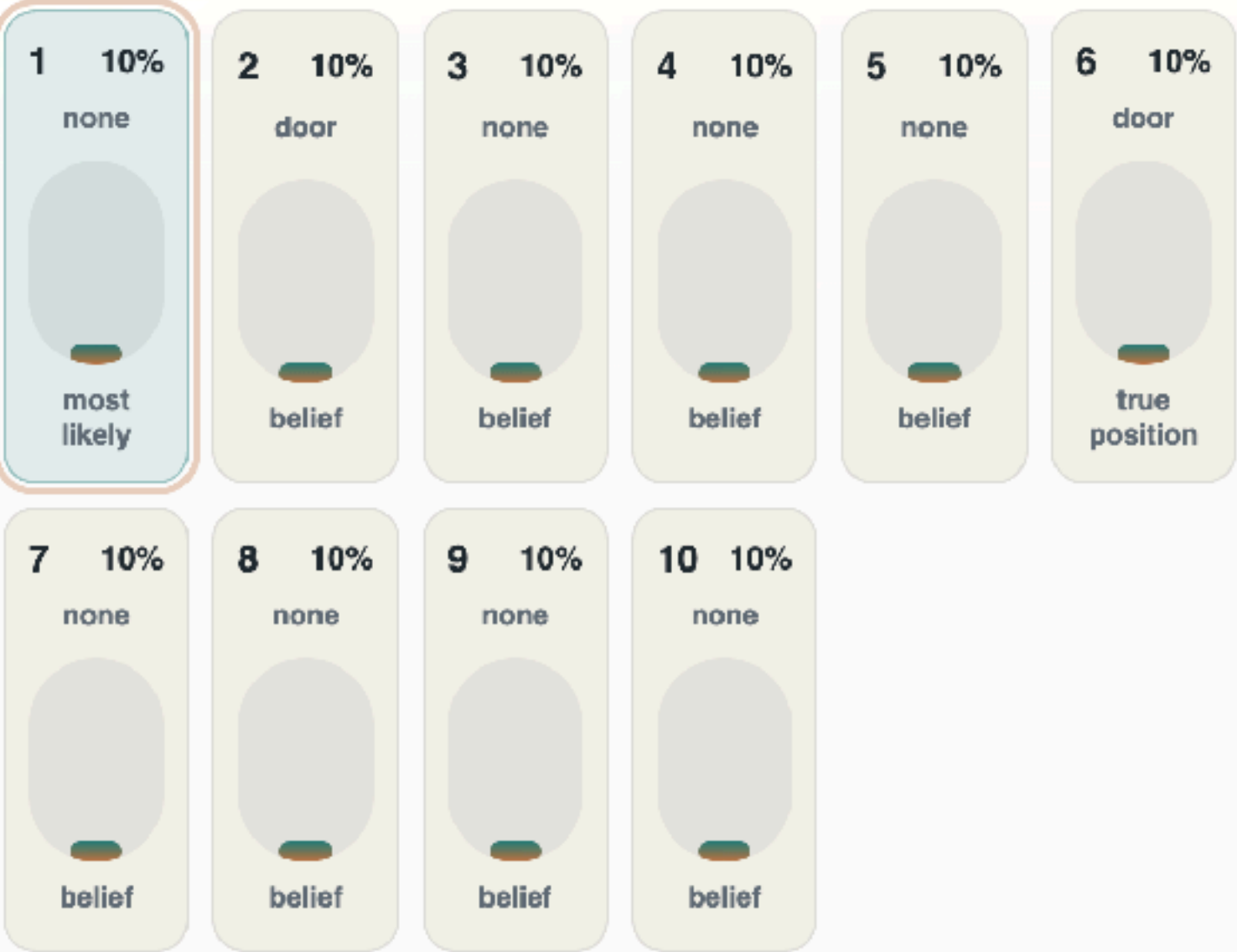
$$\text{posterior} = [0.091, 0.364, 0.091, 0.364, 0.091]$$

The observation supports cells 2 and 4, but the robot still has uncertainty.

Update Explanation

Belief values, likelihoods, normalisation, and transition details are shown for each update.

BELIEF DISTRIBUTION



Total probability: 1.000. Most likely position: cell 1, P = 0.10.

BEFORE AND AFTER VALUES

CELL	LANDMARK	PRIOR	LIKELIHOOD	UNNORMALISED	POSTERIOR
1	none	0.100	1.000	0.100	0.100
2	door	0.100	1.000	0.100	0.100
3	none	0.100	1.000	0.100	0.100
4	none	0.100	1.000	0.100	0.100
5	none	0.100	1.000	0.100	0.100
6	door	0.100	1.000	0.100	0.100
7	none	0.100	1.000	0.100	0.100
8	none	0.100	1.000	0.100	0.100
9	none	0.100	1.000	0.100	0.100
10	none	0.100	1.000	0.100	0.100

Normalisation constant: 1.000

Bayes Filter Corridor

The corridor shows landmark cells, posterior belief bars, the most likely cell, and the hidden true position when enabled.

MOTION ONLY

Move leftMove right

Prediction update. No sensor reading.

SENSING ONLY

Sense doorSense no door

Correction update. Robot stays put.

MOVE + SENSE

Move left + senseMove right + sense

Moves first, then samples door/no door using the sensor model.

Bayes Filter Corridor

Total probability 1.000 | most likely cell 1



CURRENT STATE

Operation
reset

Most likely
cell 1

Confidence
10%

ai9414

Belief over time in AI

The same idea appears in many AI systems:

- robot localisation;
- object tracking;
- sensor fusion;
- hidden Markov models;
- Kalman filters;
- particle filters;
- partially observable decision-making.

We are not teaching those methods in full today. The common idea is maintaining and updating uncertain belief over hidden state.

From planning to partial observability

Earlier planning problem (week 2):

known state $\rightarrow$ choose action $\rightarrow$ known next state

Uncertain robot problem:

hidden state $\rightarrow$ choose action $\rightarrow$ noisy observation $\rightarrow$ updated belief

This is the setting behind partially observable decision-making.

The full formal model is a POMDP (“partially observable Markov decision process”): hidden states, actions, noisy observations, and an agent that acts using a belief state.

We will not solve POMDPs here. The important idea is: plan over beliefs, not just known states.

Belief is not the final answer

The robot believes:

$$P(\text{ObjectIsCup}) = 0.7$$

$$P(\text{ObjectIsFragileOrnament}) = 0.2$$

$$P(\text{ObjectUnknown}) = 0.1$$

Question:

Should the robot pick it up?

It depends on the consequences.

Expected utility

$$EU(\text{action}) = \sum_{\text{outcomes}} P(\text{outcome}) \cdot U(\text{action}, \text{outcome})$$

- Probability says what might be true.
- Utility says how good or bad each outcome would be.
- Expected utility chooses the action with the best probability-weighted consequence.

Example: Pick up or ask for help

Belief:

$$P(\text{Cup}) = 0.7, \quad P(\text{Fragile}) = 0.2, \quad P(\text{Unknown}) = 0.1$$

Action	Cup	Fragile object	Unknown
Pick up	+10	−100	−30
Ask human	+5	+5	+5
Skip	0	0	0

The most likely class is cup, but the cost of being wrong is high.

Expected utility calculation

$$\begin{aligned}\text{EU}(\text{PickUp}) &= 0.7 \cdot 10 + 0.2 \cdot (-100) + 0.1 \cdot (-30) \\ &= 7 - 20 - 3 \\ &= -16\end{aligned}$$

$$\text{EU}(\text{AskHuman}) = 0.7 \cdot 5 + 0.2 \cdot 5 + 0.1 \cdot 5 = 5$$

$$\text{EU}(\text{Skip}) = 0$$

The best action is AskHuman, even though Cup is the most likely state.

Where does a decision threshold come from?

Suppose $p = P(\text{object is safe})$.

Action	Safe	Fragile
Pick up	+10	-100
Ask human	+2	+2

$$EU(\text{PickUp}) = 10p - 100(1 - p)$$

$$EU(\text{AskHuman}) = 2$$

Pick up only if:

$$10p - 100 \cdot (1 - p) > 2 \Rightarrow p > 0.927$$

A 0.9 confidence score is not enough under this utility model.

Thresholds are policy choices

Object detection policies might look like:

If confidence > 0.5 , pick; or

If confidence < 0.7 , ask; or

If the object may be fragile, never pick automatically.

Thresholds are not just about model accuracy.

They depend on costs, benefits, safety, workload, and the availability of more information.

Value of information

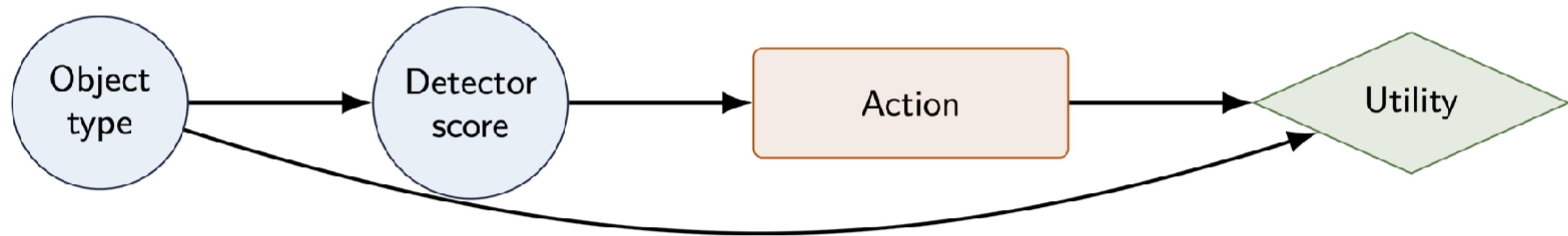
What would make the decision easier?

- Take another look / another camera image.
- Move closer.
- Ask a human.
- Use another sensor.
- Inspect from another angle.

Information has value if it can change the action.

If the robot would ask for help either way, a costly extra measurement may not be useful.

Decision network view



Chance nodes describe uncertain state. Decision nodes describe choices. Utility nodes describe consequences.

The action is chosen by the agent; it is not inferred as a hidden state.

When 90% accuracy should not act automatically

Think about examples where a model that is 90% accurate should still not act automatically.

Possible domains:

- medical diagnosis;
- pedestrian detection;
- robot grasping a fragile object;
- security decisions;
- student assessment;
- autonomous driving;
- hiring or scholarship screening.

The decision threshold depends on consequences, not just confidence.

A neural network score is not a probability

A classifier outputs:

Cat : 0.92

Dog : 0.06

Fox 0.02

Does 0.92 mean that the model is correct 92% of the time?

Only if the model is well calibrated for that setting.

Accuracy and calibration are different

Model A

90% accurate.

When it says 0.9, it is correct about 90% of the time.

Model B

90% accurate.

Often says 0.99 even when it is wrong.

Same accuracy, but different usefulness for risk-sensitive decisions.

A probability forecast can fail in different ways

For probabilistic predictions, accuracy is not the whole story.

We care about:

- **discrimination**: does the model rank likely cases above unlikely cases?
- **calibration**: when it says 0.8, is it correct about 80% of the time?
- **sharpness**: does it make useful confident predictions, rather than always saying 0.5?

A model can be accurate but overconfident. A model can be calibrated but too timid to be useful.

Reliability diagram idea

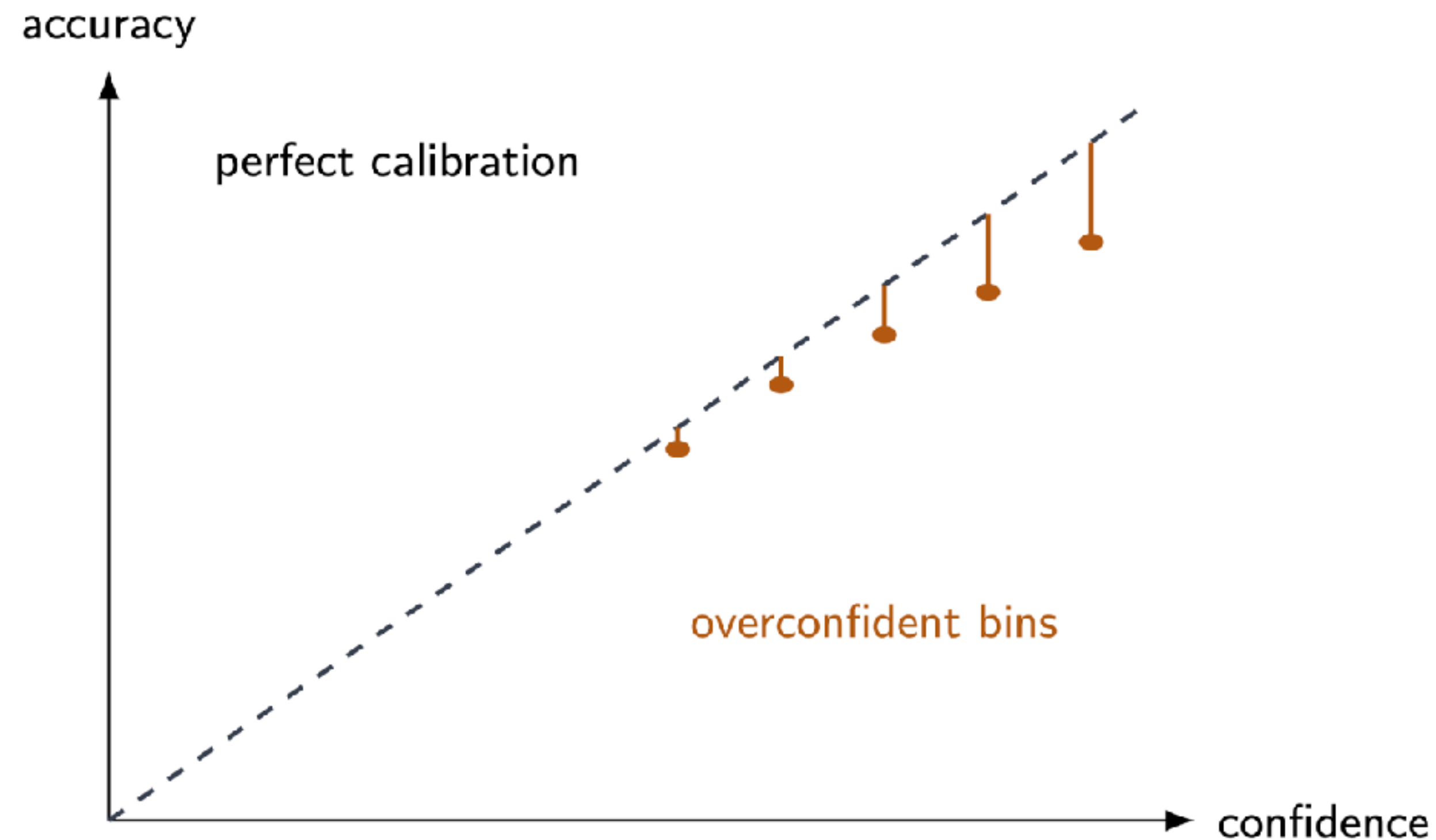
Group predictions into confidence bins and compare confidence with observed accuracy.

For each bin, compare average predicted confidence with the fraction actually correct.

Example:	Confidence bin	Expected if calibrated	Observed accuracy
	0.5 to 0.6	about 55%	52%
	0.6 to 0.7	about 65%	61%
	0.7 to 0.8	about 75%	68%
	0.8 to 0.9	about 85%	74%
	0.9 to 1.0	about 95%	81%

If observed accuracy is below confidence, the model is overconfident.

Reliability diagram visual



A reliability diagram checks whether predicted probabilities match empirical frequencies: each point summarises one confidence bin.

Sources of uncertainty in learned systems

- Noisy observations;
- limited training data;
- ambiguous input;
- distribution shift;
- Model limitations;
- Conflicting evidence
- Annotation disagreement
- Unknown unknowns.

For robotics, distribution shift is especially common: new lighting, viewpoints, objects, floor surfaces, changes in environment, and sensor conditions.

Distribution shift example

Training environment

- Well-lit lab;
- Clean floor;
- Known object set;
- Stable camera pose;
- Few occlusions

Deployment environment

- Reflective doors
- Crowded corridor
- Unseen objects
- Motion blur
- Blocked sensor view.
- ...

A high confidence score can be least trustworthy exactly when the input is unlike the data used to develop the model.

Abstention and human help

AI systems should sometimes:

- Answer
- Ask a clarifying question
- Request more evidence
- Defer to a human
- Refuse to act
- Act conservatively

Uncertainty reasoning is not only about better numbers. It is also about better behaviour under uncertainty.

Foundation model connection

Prompt:

What room should the robot enter?

A language model may produce a plausible answer even if the evidence is ambiguous.

Fluency is not calibrated belief.

This is why uncertainty, abstention, and evidence tracking matter for later topics.

Calibration by bins

Given 20 predictions, group them into bins and compare average confidence with empirical accuracy.

Confidence	Correct?	Confidence	Correct?
0.95	yes	0.78	no
0.93	no	0.76	yes
0.91	yes	0.74	no
0.88	yes	0.69	yes
0.86	no	0.66	no
0.84	yes	0.63	yes
0.82	no	0.59	no
0.81	yes	0.56	yes
0.79	no	0.54	no
0.78	yes	0.52	yes

What pattern would tell you the model is overconfident?

Calibration approach

Using bins $[0.5, 0.7)$, $[0.7, 0.8)$, $[0.8, 0.9)$, and $[0.9, 1.0]$:

Bin	Count	Average confidence	Empirical accuracy
0.5 to 0.7	7	0.60	0.57
0.7 to 0.8	5	0.77	0.40
0.8 to 0.9	5	0.84	0.60
0.9 to 1.0	3	0.93	0.67

The high-confidence bins are overconfident: their empirical accuracy is well below their average confidence.

Logic, probabilities, decisions

Question	Area
What follows if these facts are true?	Logic
What should I believe given uncertain evidence?	Probability
How should I act given uncertain belief and consequences?	Decision theory
How reliable is a learned model's confidence?	Calibration and uncertainty estimation

These are not rival views of AI. They answer different parts of the agent problem.

Important points from today

1. Uncertainty is normal in AI, not an exception.
2. Bayes' rule updates belief from evidence.
3. Bayesian networks represent structured uncertain knowledge.
4. Robots often maintain belief over hidden state.
5. Actions should be chosen using consequences, not just the most likely state.
6. Confidence scores need calibration before they can be trusted as probabilities.

Uncertainty in next weeks' topics

Computer vision

- detections are uncertain
- confidence scores can mislead
- distribution shift matters

Foundation models

- fluent answers may be wrong
- evidence tracking matters
- abstention ("I don't know") would be useful

Robotics

- state is hidden
- sensors and actions are noisy
- safe agents ask, stop, or gather more information

Question

When an AI system gives a confident answer, what evidence would make you trust it as a probability?

Look for the model, the data, the calibration, the costs, and the option to gather more information.